

EN KISA YOL ALGORITMALARI

Dijkstra Karmasikligi · Bellman-Ford · Floyd-Warshall

Kisa Hatirlatma: V = kose sayisi, E = kenar sayisi. Dijkstra: negatif kenar yok. Bellman-Ford: negatif kenar var, negatif dongu tespit eder. Floyd-Warshall: tum cifter arasi en kisa yol.

1. Dijkstra – Zaman ve Bellek Karmasikligi

Dijkstra – Zaman ve Bellek Karmasikligi

Binary Heap ile Dijkstra:

- 1 extractMin maliyeti: $\Theta(\log |V|)$
- $|V|$ tane icin toplam extractMin: $\Theta(|V| \log |V|)$
- 1 insert maliyeti: $\Theta(\log |V|)$
- $|E|$ tane icin toplam insert: $\Theta(|E| \log |V|)$

Toplam maliyet: $\Theta(\log |V|) \cdot (|E| + |V|) = \Theta((|E| + |V|) \log |V|)$

Fibonacci Heap ile Dijkstra:

- Fibonacci heap maliyetleri: $\Theta(1)$ (insert ve decrease-key)
- $|E|$ kenar icin: $\Theta(|E|)$ toplam
- $|V|$ extractMin icin: $\Theta(|V| \log |V|)$ toplam

Fibonacci Heap toplam maliyet: $\Theta(|E| + |V| \log |V|)$

Neden daha iyi? Yogun graflar icin ($|E| \approx |V|^2$): Binary Heap $O(|V|^2 \log |V|)$ yaparken Fibonacci $O(|V|^2)$ yapar.

Bellek Karmasikligi: $\Theta(|V|)$

Heap Turu	extractMin	insert/decrease	Toplam
Binary Heap	$\Theta(\log V)$	$\Theta(\log V)$	$\Theta((E + V) \log V)$
Fibonacci Heap	$\Theta(\log V)$ amort.	$\Theta(1)$ amort.	$\Theta(E + V \log V)$

Pratik – Dijkstra Karmasikligi

(a) $|V| = 100$, $|E| = 5000$ için Binary Heap ve Fibonacci Heap toplam maliyet degerlerini yaklasik hesaplayin. ($\log 100 \approx 7$) _____

(b) Seyrek bir grafta ($|E| \approx |V|$) hangi heap daha avantajlidir? _____

2. Bellman-Ford Algoritmasi

Bellman-Ford dinamik programlama yaklasimini kullanir. Negatif agirlikli yonlu graftlar icin en kisa yolu bulabilir. Negatif donguleri de tespit eder.

Bellman-Ford Algoritmasi – Dinamik Programlama**Calisma Adimlari:**

1. Kaynak kose icin baslangic mesafesini 0'a ayarla. Diger tum koseler icin mesafeleri ∞ 'a ayarla.
2. Dongusu olmayan en uzun kisa yol en fazla $|V| - 1$ kenara sahip oldugu icin tum kenarlari $|V| - 1$ kez kontrol et. (Kontrol islemi icin bitisiklik matrisi [adjacency matrix] kullanilir.) Her kenar icin daha kisa bir mesafenin hesaplanip hesaplanmay-acagini kontrol et ve hesaplanan mesafe daha kisaysa mesafeyi guncelle.
3. 1 kez daha tara. Eger hala guncelleme oluyorsa grafta **negatif dongu var** demektir. Bu durumda algoritma dogru sonuc uretmez.

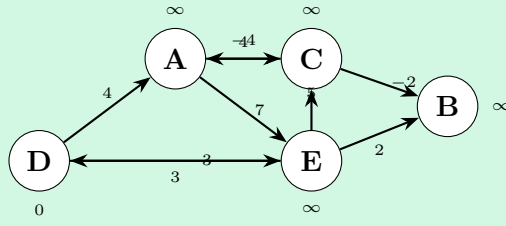
Bellman-Ford Ana Dongusu (Relax):

```
for (int i = 1; i <= V; i++) {
    for (int j = 0; j < E; j++) {
        int u = graph.edges[j].src;
        int v = graph.edges[j].dest;
        int w = graph.edges[j].weight;
        if (dist[u] != INF && dist[u] + w < dist[v])
            dist[v] = dist[u] + w;
    }
}
```

Negatif Dongu Kontrolu:

```
for (int j = 0; j < E; j++) {
    int u = graph.edges[j].src;
    int v = graph.edges[j].dest;
    int w = graph.edges[j].weight;
    if (dist[u] != INF && dist[u] + w < dist[v]) {
        System.out.println("Negatif dongu tespit edildi!");
        return;
    }
}
```

Ornek Graf ve Iterasyon Tablosu (D' den baslangic):



Iter.	A	B	C	D	E
Bas.	∞	∞	∞	0	∞
1	4	5	1	0	3
2	-2	5	1	0	3
3	-2	5	1	0	3
4	-2	5	1	0	3

$V = \{(D, A), (D, C), (D, E), (A, E), (A, C), (C, A), (E, C), (E, B), (B, C)\} - |V| - 1 = 4$ kez

Avantajlari:

- Negatif dongulerin olmadigi negatif agirlikli yonlu graflar ele alinabilir.
- Yonlu ve yonsuz negatif olmayan agirlikli graflar icin de calisir.
- Negatif dongulerin varligini kontrol etmek icin kullanilabilir.

Dezavantajlari:

- Dijkstra algoritmasindan daha yuksek zaman karmasikligina sahiptir. Bu nedenle uygulanabilir alani Dijkstra'ya kiyasla daha azdir.

Karmasiklik:

Zaman: $\Theta(|E| \cdot |V|)$

Bellek: $\Theta(|V|)$

Pratik – Bellman-Ford

(a) $|V| = 5$, $|E| = 9$ olan ornekte Bellman-Ford toplam kac kenar relax islemi yapar?

(b) Ornekte neden 2. iterasyonda $A = -2$ oluyor? Hangi kenar bunu sagliyor, hangi degerleri kullaniyor? _____

(c) 5. iterasyonda (yani $|V|$. tur) hala guncelleme olsaydi ne anlama gelirdi? _____

3. Floyd-Warshall Algoritmasi

Bir graftaki tum dugum ciftleri arasindaki en kisa yolari bulur. Dinamik programlama kullanir. Negatif kenarlari destekler, negatif donguleri tespit eder.

Floyd-Warshall Algoritmasi

Calisma Adimlari:

1. Her dugumden kendisine mesafe 0'dir. Dogrudan bagli dugumler icin kenar agirlikini yaz. Baglanti yoksa mesafeyi ∞ kabul et.

2. Her dugumu sirayla **ara durak** olarak sec. Ara durak uzerinden gecen tum (i, j) ciftlerini kontrol et ve mesafeleri guncelle. Her yeni ara durakla bir onceki iterasyonda bulunan kisa yollarin uzerine insa edilir. Boylece ilkden fazla ara durakli yollar da parca parca kesfedilir. Bu tum dugumler icin tekrarla:

$$\text{dist}[i][j] > \text{dist}[i][k] + \text{dist}[k][j] \Rightarrow \text{dist}[i][j] = \text{dist}[i][k] + \text{dist}[k][j]$$

3. Eger bir dugumun kendisine mesafesi negatifse **negatif dongu var** demektir. Cunku bir dugumun kendisine mesafesi normalde 0'dir.

Floyd-Warshall Kodu (Uc Ic Ice Dongu):

```
// k: ara durak, i: kaynak, j: hedef
for (int k = 0; k < V; k++) {
    for (int i = 0; i < V; i++) {
        for (int j = 0; j < V; j++) {
            if (dist[i][k] + dist[k][j] < dist[i][j])
                dist[i][j] = dist[i][k] + dist[k][j];
        }
    }
}
// Negatif dongu kontrolu
for (int i = 0; i < V; i++)
    if (dist[i][i] < 0)
        System.out.println("Negatif dongu var!");
```

Avantajlari:

- Bir graftaki dugumler arasindaki en kisa yolulari bulmanin disinda bir grafın gecisli kapanmasini (transitive closure) hesaplamak ve bir graftaki negatif donguleri tespit etmek icin de kullanilir.

Dezavantajlari:

- Bellek karmasikligi yuksektir: $\Theta(V^2)$ matris gerekir.
- Yonsuz graflar icin verimli degildir.

Karmasiklik:

Zaman: $\Theta(V^3)$

Bellek: $\Theta(V^2)$

Kod: iceice 3 dongu, notasyon S

Pratik – Floyd-Warshall

(a) $V = \{A, B, C\}$, $E = \{(A, B, 3), (B, C, -1), (A, C, 5)\}$ grafında Floyd-Warshall ile A 'dan C 'ye en kisa yolu bulun. ($k = B$ ara duragu ne saglar?) _____

(b) Floyd-Warshall'in negatif dongu tespit mekanizmasini aciklayin. Hangi dizine bakiliyor ve neden? _____

(c) $|V| = 100$ için Floyd-Warshall kac islem yapar? Bellman-Ford ile ($|E| = 500$) karsilastirin. _____

Karsilastirma Ozeti:

Algoritma	Zaman	Bellek	Neg. ke- nar?	Ne icin?
Dijkstra (Binary)	$O((V + E) \log V)$	$O(V)$	Hayir	Tek kaynak, hizli
Dijkstra (Fibonacci)	$O(E + V \log V)$	$O(V)$	Hayir	Yogun graf
Bellman-Ford	$O(V \cdot E)$	$O(V)$	Evet	Negatif kenar, dongu det.
Floyd-Warshall	$O(V^3)$	$O(V^2)$	Evet	Tum cifter arasi